

3D Fluid Simulation in CUDA

Lasse Henrich

Technical University Berlin

Introduction

This project aims to simulate the dynamics of fluids, which is the basis for simulating a variety of natural phenomena, like ocean waves, smoke, fire, cloud formations, shockwaves, or oil spills. The physics of how liquids and gases move are governed by the Navier-Stokes equations. Fluid simulation is performed on a grid of cells and well suited for parallel computation.

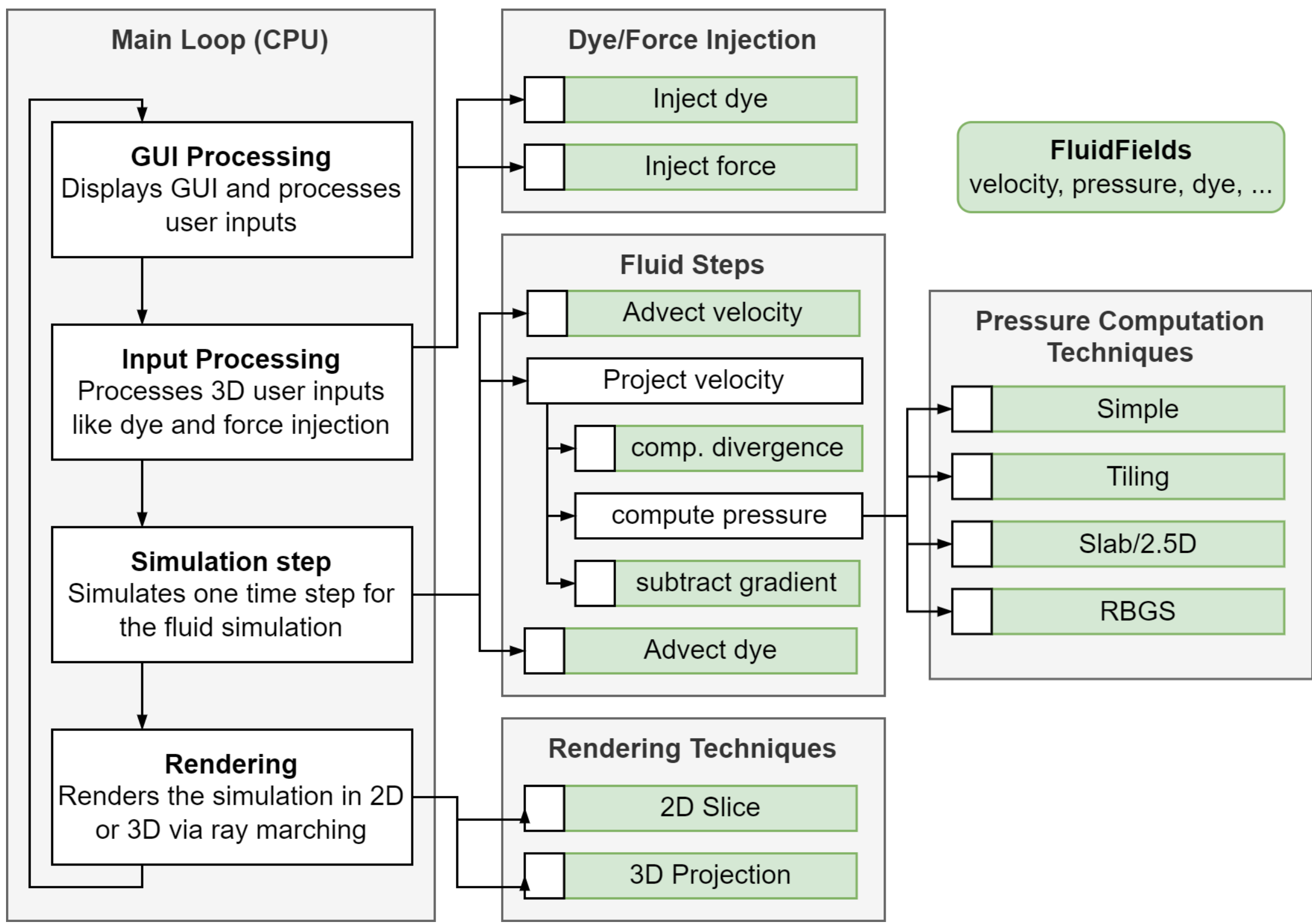
A fluid is represented by its velocity field v and a scalar pressure field p , which vary in space and time. If v for $t = 0$ is known and *incompressibility* is assumed, the fluid's state over time can be described by the *Navier-Stokes* equations for constant-density incompressible flow, which are generally governed by advection: **Advection** is the transfer of a property from one place to another due to the motion of the fluid. If you put dye into moving water, the dye will be pulled (*advected*) through it. The velocity of a fluid carries *itself* along as well, which is called *self-advection*. Note that this simulation currently assumes *zero viscosity*.

Implementation Overview

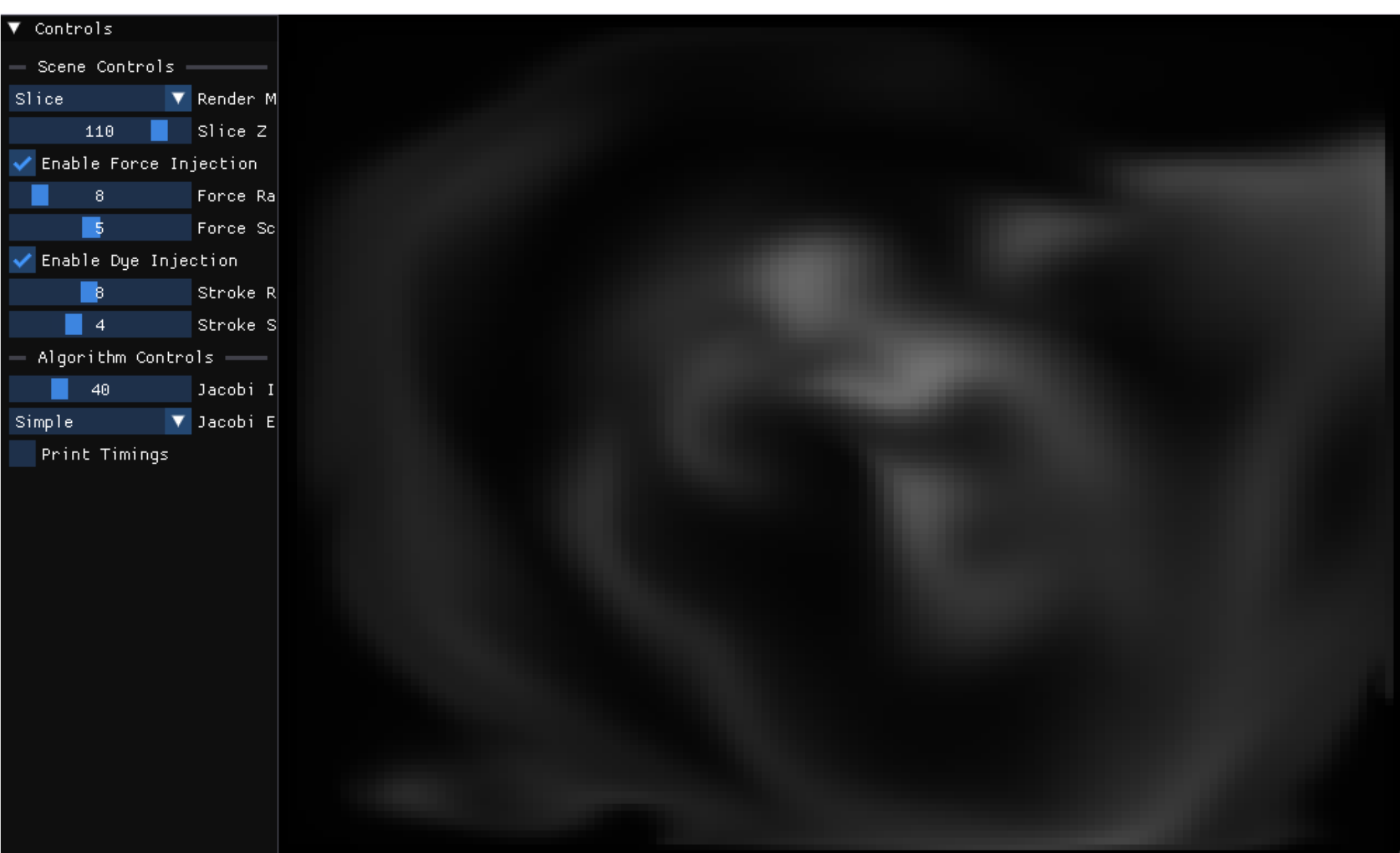
The main loop runs without end. It first renders the GUI and processes user inputs (like selecting a different rendering mode). Then, 3D input is processed: If the user clicks and drags in the scene, dye and force are injected into the simulation grid.

The actual simulation is divided into (i) velocity advection, (ii) velocity projection, and (iii) dye advection. Profiling revealed that the pressure calculation for projecting the velocity largely bottlenecks the whole application's performance, so four different methods were implemented that all approximate the same pressure.

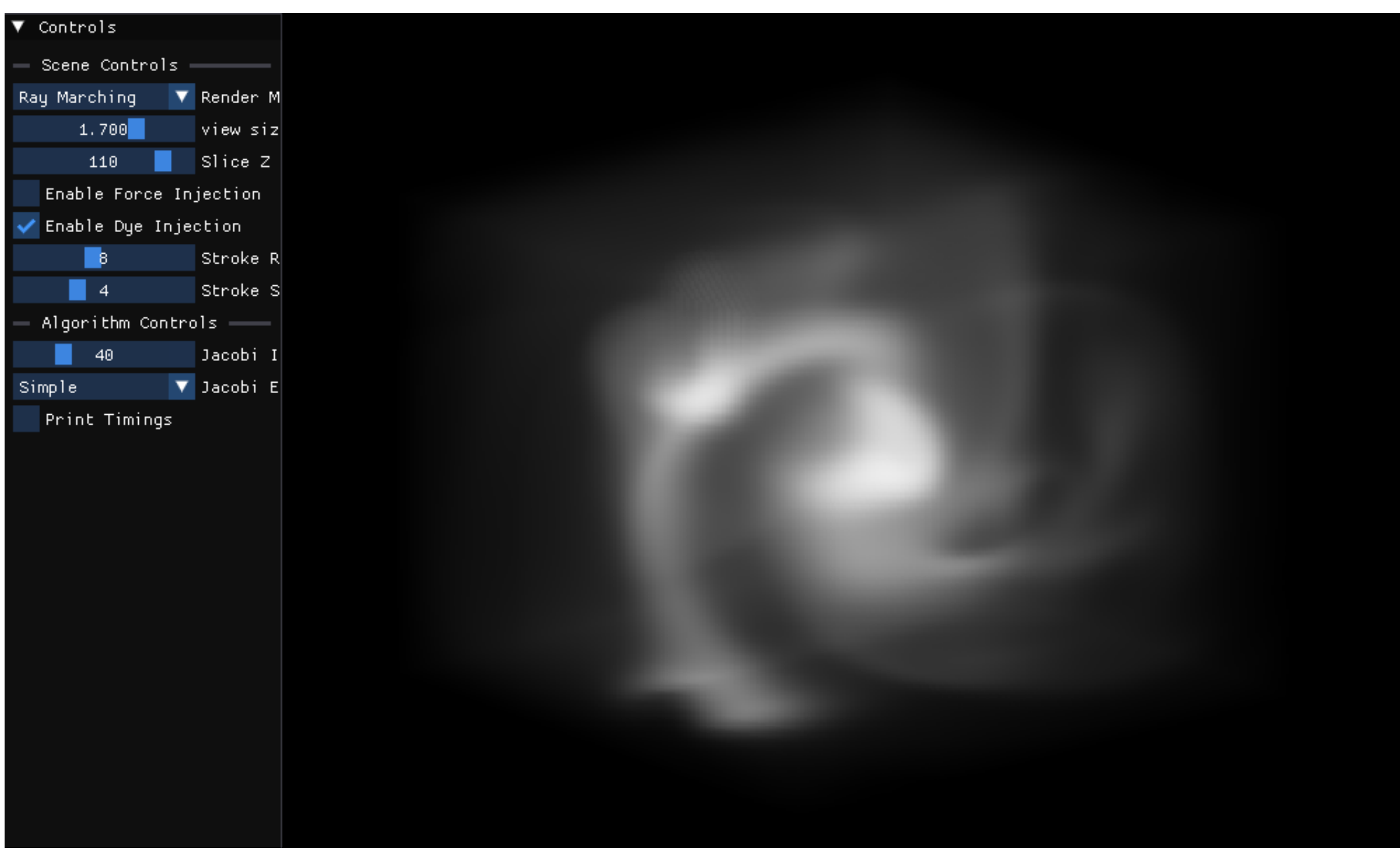
Lastly, the simulated 3D grid is visualized, either by rendering just a two-dimensional slice of it, or by rendering the entire cube via ray marching, orthographically projecting it onto the screen with a rotating camera.



Demo



2D Slice View



3D Orthographic Projection

References

- [1] Mark J. Harris. Fast fluid dynamics simulation on the GPU. In Randima Fernando, editor, GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics, chapter 38, pages 637–665. Addison-Wesley Professional, 2004.
- [2] Jos Stam. Stable fluids. In Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '99, pages 121–128, New York, NY, USA, 1999. ACM Press/Addison-Wesley Publishing Co.
- [3] Jos Stam. Real-time fluid dynamics for games. In Proceedings of the Game Developers Conference (GDC), pages 1–25, 2003.
- [4] Jamie Wong. WebGL fluid simulation, aug 2016.

Simulation Steps

The simulation consists of the steps **velocity advection** → **velocity projection** → **dye advection**.

1. Velocity and Dye advection

At position x , the new value at time $t + \delta t$ of quantity q is calculated by tracing that quantity back in time:

$$q(x, t + \delta t) = q(x - v(x, t)\delta t, t).$$

Instead of pushing a quantity (dye or velocity) forward, each cell checks where its fluid came from and **pulls** the quantity from there. That means that *each thread writes exactly one cell and only reads others* → no race conditions that would need to be solved. Also, since the *source position* mostly lands between cell centers, the eight surrounding cells are trilinearly interpolated.

2. Velocity projection

For a constant-density incompressible fluid, conservation of mass requires the velocity field to be **divergence-free**: $\nabla \cdot v = 0$. However, after velocity advection and external forces are applied, it's likely not; instead, the current velocity field $v* = v + \nabla p$ is the sum of a divergence-free (incompressible) velocity field v and the divergent parts, which is the gradient of the pressure field ∇p . Therefore, the pressure gradient needs to be subtracted, where pressure p is calculated with a Jacobi iteration repeatedly applying

$$p_{x,y,z}^{(k+1)} = \frac{p_{x-1,y,z}^{(k)} + p_{x+1,y,z}^{(k)} + p_{x,y-1,z}^{(k)} + p_{x,y+1,z}^{(k)} + p_{x,y,z-1}^{(k)} + p_{x,y,z+1}^{(k)} - \nabla \cdot v*}{6}.$$

For this, the divergence $\nabla \cdot v* = \frac{\partial v*_x}{\partial x} + \frac{\partial v*_y}{\partial y} + \frac{\partial v*_z}{\partial z}$ is calculated with finite differences:

$$\nabla \cdot v* = 0.5 \cdot ((v_{x,x+1}^* - v_{x,x-1}^*) + (v_{y,y+1}^* - v_{y,y-1}^*) + (v_{z,z+1}^* - v_{z,z-1}^*)).$$

After having calculated the pressure p from the divergence, its gradient ∇p is computed (again with finite differences) and subtracted from the velocity, i.e. $v = v* - \nabla p$, using a third kernel. The three principal projection kernels perform **output-centric** decompositions via **stencil computation**.

Since pressure computation is performance-heavy, it was implemented with four methods, three of which are performance optimizations over the naive solution, which executes 40 Jacobi iterations and does not respect any memory utilization best practices. Since the *Iterative Stencil Loop* (ISL) could lead to race conditions, **double buffering (ping-pong)** with a pointer swap is implemented, i.e. reading from buffer A and writing to buffer B, then swapping the buffers in $\mathcal{O}(1)$ time.

Optimization 1: Standard Tiling:

- A *tile* (plus a *halo* region) is loaded into shared memory collaboratively by the threads of a block, **improving data locality and reducing memory access overhead**.
- Decreases performance in 3D, since thread utilization goes down dramatically: CUDA's 1024-thread block limit of 10^3 results in 51%.

Optimization 2: Slab/2.5D:

- Every block is now responsible for a 2D tile, which is moved through the z -dimension of the grid. Each thread does N times the work now → **Coarsening**.
- Further, in 2D, the x-dimension can now be set to 32, filling a whole warp. A 32^2 -sized tile is possible and would result in an 88% output fraction, but 32×8 exhibits better performance.

Optimization 3: Red-Black Gauss Seidel:

- RBGS splits the grid into a **3D checkerboard** of *red* and *black* cells. Per iteration, first all red cells are updated using the black neighbors, then vice versa.
- Because RBGS immediately reuses newly updated values, its convergence rate roughly doubles. The implementation therefore performs **half as many iterations** for roughly the same accuracy.
- All red and all black cells are packed into their own contiguous memory addresses, improving **coalescing**.

Rendering Techniques

2D Slice: Most simply, this technique takes a slice from the dye grid at a specified z and prints it to an OpenGL texture.

3D Projection via Ray Marching: A *camera* position and rotation are adjusted each frame, rotating around the simulation grid. For one position/rotation, a kernel generates parallel rays in an **orthographic projection** and uses the *Slab Method* to calculate where a ray enters and exits the grid. Marching along the ray, dye is sampled and brightness for that pixel is accumulated via the *Beer-Lambert law*. That brightness gets clamped and written to an OpenGL texture.